Мастерская MusicTech — Направление 1: Логика работы

Score-Following

Алгоритмы отслеживания партитуры в реальном времени

Никита Новицкий

Центральный Университет (Т-Банк)

2026

Содержание

1 Введение **3**
- 1.1 Что такое score-following . . . 3
- 1.2 Области применения . . . 3
- 1.3 Краткая история . . . 3
- 1.4 Pipeline системы score-following . . . 4

2 MIDI и музыкальные основы **4**
- 2.1 Протокол MIDI . . . 4
- 2.2 Таблица нот MIDI . . . 5
- 2.3 Клавиатура и MIDI-номера . . . 5
- 2.4 Представление партитуры и исполнения . . . 5

3 Формализация задачи **5**
- 3.1 Функция выравнивания . . . 5
- 3.2 Ограничение монотонности . . . 6
- 3.3 Функция стоимости . . . 6
- 3.4 Оптимальное выравнивание . . . 6
- 3.5 Online vs Offline . . . 6
- 3.6 Визуализация выравнивания . . . 6

4 Dynamic Time Warping (DTW) **7**
- 4.1 Определение . . . 7
- 4.2 Матрица локальной стоимости . . . 7
- 4.3 Рекурсия DTW . . . 7
- 4.4 Оптимальный путь . . . 8
- 4.5 Псевдокод DTW . . . 8
- 4.6 Сложность . . . 8
- 4.7 Ограничение Sakoe–Chiba . . . 9
- 4.8 Визуализация: матрица стоимости и оптимальный путь . . . 9
- 4.9 Числовой пример . . . 9

5 Online Time Warping (OLTW) **10**
- 5.1 Определение . . . 10
- 5.2 Инкрементальное обновление . . . 10
- 5.3 Механизм RunCount . . . 10
- 5.4 Псевдокод OLTW . . . 11
- 5.5 Сложность . . . 12
- 5.6 Визуализация: пошаговое построение пути . . . 12
- 5.7 Сравнение DTW и OLTW . . . 12

6 Скрытые марковские модели (HMM) **12**
- 6.1 HMM для score-following . . . 12
- 6.2 Параметры модели . . . 13
  - 6.2.1 Матрица переходов A . . . 13
  - 6.2.2 Функция эмиссии B . . . 13

6.2.3 Начальное распределение π . . . 13
6.3 Алгоритм Forward . . . 14
6.4 Алгоритм Viterbi . . . 14
6.5 Диаграмма состояний HMM . . . 14
6.6 Псевдокод алгоритмов . . . 15
6.7 Числовой пример . . . 16
6.8 Связь с марковскими цепями . . . 16
6.9 HMM с зависимостью от длительности . . . 16

7 Метрики качества **17**
7.1 Ошибка выравнивания (Alignment Error) . . . 17
7.2 Точность при пороге (Precision at Threshold) . . . 17
7.3 Латентность . . . 17
7.4 Доля пропусков (Miss Rate) . . . 18
7.5 Степень завершения (Piece Completion Rate) . . . 18
7.6 Сравнение типичных значений метрик . . . 18
7.7 Визуализация ошибки выравнивания . . . 18

8 Практический пример **18**
8.1 Описание данных . . . 18
8.2 Партитура и исполнение . . . 19
8.3 Кривая темпа . . . 19
8.4 Распределение высот . . . 20
8.5 DTW: матрица стоимости . . . 20
8.6 DTW: оптимальный путь . . . 21
8.7 HMM: вероятности Forward . . . 22
8.8 Viterbi: оптимальный путь . . . 23
8.9 OLTW: отслеживание в реальном времени . . . 23
8.10 Сравнение латентности . . . 24
8.11 Кривые точности при разных порогах . . . 24
8.12 Итоги сравнения . . . 25

9 Нейросетевые подходы **25**
9.1 Обзор современных методов . . . 25
9.1.1 CNN для audio-to-score alignment . . . 25
9.1.2 Transformer-подходы . . . 25
9.1.3 HeurMiT (2025) . . . 25
9.2 Преимущества и недостатки . . . 26

10 Выводы и перспективы **26**
10.1 Резюме алгоритмов . . . 26
10.2 Рекомендация для проекта MusicTech . . . 27
10.3 Связь с проектом переменных звёзд . . . 27
10.4 Дальнейшая работа . . . 27

1 Введение

1.1 Что такое score-following

Score-following (отслеживание партитуры) — это задача автоматического определения текущей позиции живого исполнителя в нотном тексте (партитуре) в реальном времени. Система принимает на вход поток музыкальных событий — MIDI-ноты или аудиосигнал — и выдаёт соответствие между тем, что играет исполнитель, и тем, что написано в нотах.

Живое исполнение никогда не совпадает с партитурой точно: музыкант допускает отклонения в темпе (*rubato*), динамике, может пропускать ноты или добавлять орнаменты. Задача алгоритма — быть устойчивым к этим отклонениям и *в реальном времени* указывать, какую ноту исполнитель играет прямо сейчас.

1.2 Области применения

1. **Автоматический аккомпанемент.** Компьютер «следит» за солистом и подстраивает оркестровую партию под его темп. Первые такие системы появились в 1984 году.
2. **Автоматический переворот страниц.** Электронный пульт (например, iPad с нотами) автоматически перелистывает страницу, когда исполнитель доходит до конца.
3. **Визуализация в реальном времени.** Подсветка текущей ноты в партитуре для зрителей, синхронизация световых эффектов с музыкой.
4. **Музыкальное образование.** Оценка точности исполнения ученика: где он ошибся, где отстал от темпа, какие ноты пропустил.
5. **Интерактивные инсталляции.** Генеративная графика и звук, реагирующие на живое исполнение.

1.3 Краткая история

Задача score-following была поставлена независимо двумя исследователями в 1984 году: Роджер Данненберг (Roger Dannenberg, Carnegie Mellon) [1] и Барри Версо (Barry Vercoe, MIT) [2]. Оба предложили алгоритмы на основе динамического программирования для отслеживания MIDI-клавиатуры.

В 1990–2000-х годах доминировали подходы на основе *скрытых марковских моделей* (HMM): Orio & Déchelle [6], Raphael [7]. Параллельно развивались методы *Dynamic Time Warping* (DTW): Dixon [5], Müller [11].

С 2018 года начали появляться нейросетевые подходы: Dorfer et al. [9] применили свёрточные сети для audio-to-score alignment, а в 2025 году Henkel et al. [10] представили HeurMiT — нейросетевой фреймворк для полифонических инструментов.

Постановка задачи score-following

Дано:

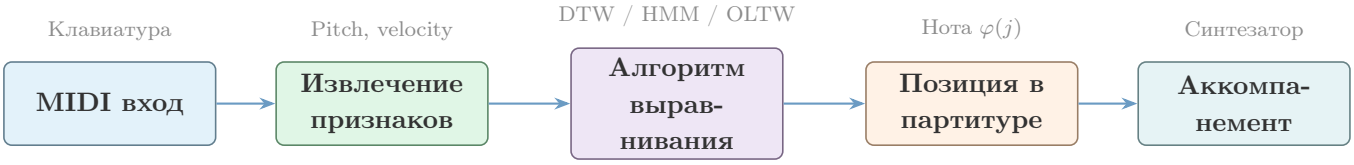
- Партитура $S = \{(p_i,\ t_i^{\text{on}},\ t_i^{\text{off}})\}_{i=1}^{N}$ — последовательность нот с высотами p_i и временными метками.
- Поток событий исполнения $P = \{(q_j,\ \tau_j^{\text{on}},\ \tau_j^{\text{off}})\}_{j=1}^{M}$, поступающий в реальном времени.

Найти: функцию выравнивания $\varphi : \{1, \ldots, M\} \to \{1, \ldots, N\}$, такую что $\varphi(j)$ — позиция в партитуре, соответствующая j-му событию исполнения.

Требования:

- *Реальное время:* $\varphi(j)$ вычисляется, используя только $p_1, \ldots, p_j$ (без знания будущих событий).
- *Монотонность:* если $j_1 < j_2$, то $\varphi(j_1) \leq \varphi(j_2)$.
- *Робастность:* устойчивость к ошибкам, пропускам, rubato.

1.4 Pipeline системы score-following



2 MIDI и музыкальные основы

2.1 Протокол MIDI

MIDI (*Musical Instrument Digital Interface*) — это стандарт цифровой передачи музыкальных данных, разработанный в 1983 году. В отличие от аудиосигнала, MIDI передаёт не звуковую волну, а *символические события*: какую ноту нажали, с какой силой, когда отпустили.

Основные типы сообщений, релевантные для score-following:

- `Note On` (`0x90`) — нажатие клавиши. Содержит номер ноты (pitch) и силу нажатия (velocity).
- `Note Off` (`0x80`) — отпускание клавиши. Содержит номер ноты и velocity отпускания (обычно 0).
- `Control Change` (`0xB0`) — педаль, модуляция и др.
- `Program Change` (`0xC0`) — смена инструмента.

MIDI-событие

Каждое MIDI-событие представляется кортежем:

$$e = (\mathit{type},\ \mathit{pitch},\ \mathit{velocity},\ \mathit{timestamp})$$

где:

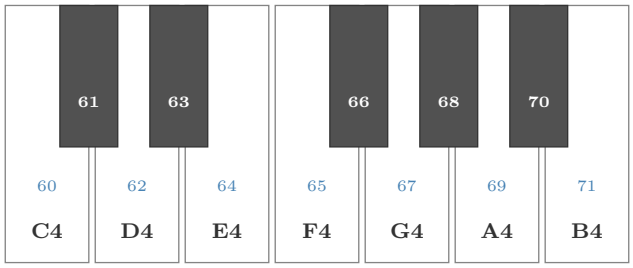
- $\mathit{type} \in \{\texttt{note_on}, \texttt{note_off}\}$
- $\mathit{pitch} \in \{0, 1, \ldots, 127\}$ — номер ноты по стандарту MIDI
- $\mathit{velocity} \in \{0, 1, \ldots, 127\}$ — сила нажатия
- $\mathit{timestamp} \in \mathbb{R}_{\geq 0}$ — время в секундах

2.2 Таблица нот MIDI

Нота	C3	D3	E3	F3	G3	A3	B3	C4	D4	E4	F4	G4	A4
MIDI	48	50	52	53	55	57	59	60	62	64	65	67	69

Таблица 1: Соответствие нот и номеров MIDI. C4 («до» первой октавы) = 60.

2.3 Клавиатура и MIDI-номера



2.4 Представление партитуры и исполнения

Для алгоритмов score-following нам нужны два формальных представления:

Партитура (Score)

Последовательность нотных событий:

$$S = \{(p_i,\ t_i^{\text{on}},\ t_i^{\text{off}})\}_{i=1}^{N}$$

где $p_i \in \{0, \ldots, 127\}$ — высота i-й ноты, t_i^{on} и t_i^{off} — номинальные времена начала и конца ноты по партитуре. N — общее количество нот в произведении.

Исполнение (Performance)

Последовательность реальных событий от исполнителя:

$$P = \{(q_j,\ \tau_j^{\text{on}},\ \tau_j^{\text{off}})\}_{j=1}^{M}$$

где q_j — реально сыгранная высота, τ_j^{on} и τ_j^{off} — реальные времена. M может отличаться от N: исполнитель может пропускать или добавлять ноты.

3 Формализация задачи

3.1 Функция выравнивания

Функция выравнивания

Задача score-following — построить отображение:

$$\varphi : \{1, \ldots, M\} \longrightarrow \{1, \ldots, N\}$$

которое сопоставляет каждому событию исполнения j позицию в партитуре $\varphi(j)$.

3.2 Ограничение монотонности

Ключевое ограничение: исполнитель играет партитуру «вперёд» (с возможными пропусками, но без возврата назад):

$$j_1 < j_2 \implies \varphi(j_1) \leq \varphi(j_2)$$

Равенство $\varphi(j_1) = \varphi(j_2)$ допускается: несколько событий исполнения могут относиться к одной ноте партитуры (например, повторные нажатия, трели, орнаменты).

3.3 Функция стоимости

Функция стоимости (cost function)

Для сравнения ноты партитуры s_i и события исполнения p_j определим локальную стоимость:

$$d(s_i, p_j) = |p_i - q_j|$$

где p_i — pitch ноты в партитуре, q_j — pitch сыгранной ноты.
Если pitch совпадает: $d = 0$. Полутоновая ошибка: $d = 1$. Октавная ошибка: $d = 12$.

3.4 Оптимальное выравнивание

Оптимальное выравнивание минимизирует суммарную стоимость:

Оптимальное выравнивание

$$\varphi^* = \arg\min_{\varphi} \sum_{j=1}^{M} d\big(s_{\varphi(j)},\ p_j\big)$$

при ограничении монотонности φ.

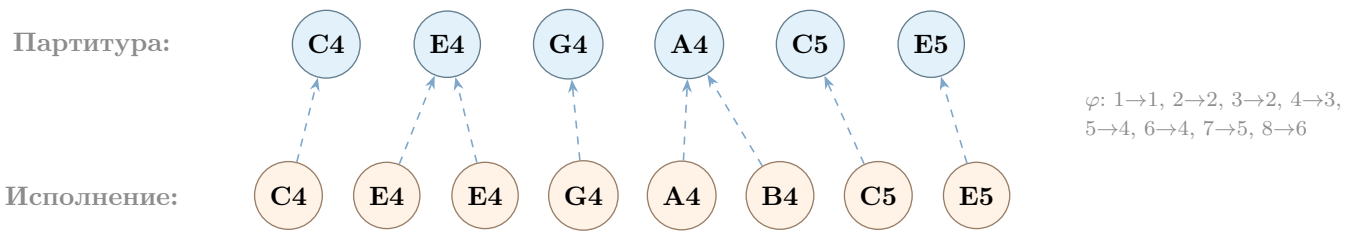
3.5 Online vs Offline

Принципиальное различие двух постановок:

- **Offline:** доступно всё исполнение P целиком. Можно вычислить глобально оптимальное φ^*. Применение: анализ записей, разметка данных.
- **Online:** в момент j доступны только $p_1, \ldots, p_j$. Нужно выдать $\varphi(j)$ немедленно, без знания будущих событий. Применение: живой аккомпанемент, переворот страниц.

В проекте MusicTech нас интересует именно *online*-постановка.

3.6 Визуализация выравнивания



На диаграмме видно, что события 3 и 6 исполнения связываются с нотами 2 и 4 партитуры соответственно (повторные нажатия). Нота B4 (событие 6) выравнивается к A4 — это ошибка исполнителя, и алгоритм выбирает ближайшую ноту партитуры.

4 Dynamic Time Warping (DTW)

4.1 Определение

Dynamic Time Warping

DTW (*Dynamic Time Warping*, динамическая трансформация временной шкалы) [3] — это алгоритм поиска оптимального выравнивания двух временных последовательностей, допускающий нелинейные деформации по времени.
Даны две последовательности:

$$X = (x_1, x_2, \ldots, x_N), \quad Y = (y_1, y_2, \ldots, y_M)$$

DTW находит путь через матрицу стоимости, минимизирующий суммарное расстояние между выровненными элементами.

4.2 Матрица локальной стоимости

Для каждой пары (i, j) вычисляем локальную стоимость:

$$C(i, j) = d(x_i, y_j)$$

В случае MIDI-нот: $d(x_i, y_j) = |p_i - q_j|$ (разница pitch). Для audio: евклидово расстояние между спектральными векторами.

Матрица C имеет размер $N \times M$.

4.3 Рекурсия DTW

Рекурсия DTW

Матрица накопленной стоимости D вычисляется рекурсивно:

$$D(i, j) = C(i, j) + \min \begin{cases} D(i-1,\ j) & \text{(вставка)} \\ D(i,\ j-1) & \text{(удаление)} \\ D(i-1,\ j-1) & \text{(совпадение)} \end{cases}$$

с граничными условиями:

$$D(0, 0) = 0, \quad D(i, 0) = \infty \ \forall\, i > 0, \quad D(0, j) = \infty \ \forall\, j > 0$$

Три варианта перехода имеют интуитивный смысл:

- $D(i-1, j-1) \to D(i, j)$: диагональный шаг — следующая нота партитуры сопоставляется со следующей нотой исполнения.
- $D(i-1, j) \to D(i, j)$: вертикальный шаг — нота партитуры пропущена исполнителем.

- $D(i, j-1) \to D(i, j)$: горизонтальный шаг — исполнитель сыграл лишнюю ноту (орнамент, ошибка).

4.4 Оптимальный путь

Итоговая стоимость выравнивания: $D(N, M)$. Оптимальный путь $W = (w_1, w_2, \ldots, w_K)$, где $w_k = (i_k, j_k)$, находится обратным проходом (*backtracking*) от (N, M) к $(1, 1)$: на каждом шаге выбираем предшественника с минимальным значением D.

4.5 Псевдокод DTW

Алгоритм DTW

```
def dtw(X, Y):
    N, M = len(X), len(Y)
    D = matrix(N+1, M+1, fill=inf)
    D[0][0] = 0

    for i in range(1, N+1):
        for j in range(1, M+1):
            cost = abs(X[i-1] - Y[j-1])
            D[i][j] = cost + min(
                D[i-1][j],       # insertion
                D[i][j-1],       # deletion
                D[i-1][j-1]      # match
            )

    # backtracking
    path = [(N, M)]
    i, j = N, M
    while i > 1 or j > 1:
        candidates = []
        if i > 1: candidates.append((D[i-1][j], i-1, j))
        if j > 1: candidates.append((D[i][j-1], i, j-1))
        if i > 1 and j > 1:
            candidates.append((D[i-1][j-1], i-1, j-1))
        _, i, j = min(candidates)
        path.append((i, j))

    return D[N][M], reversed(path)
```

4.6 Сложность

Сложность DTW

- **Время:** $O(N \cdot M)$
- **Память:** $O(N \cdot M)$

Для произведения из $N = 1000$ нот и исполнения из $M = 1100$ нот: $\approx 1{,}1 \cdot 10^6$ операций — мгновенно для offline.

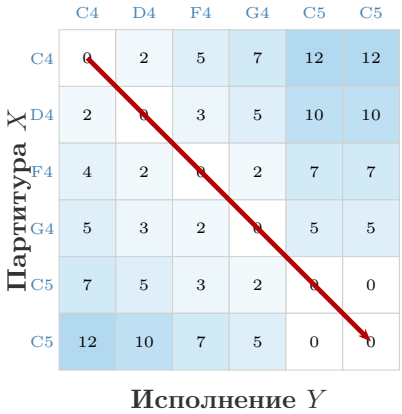
4.7 Ограничение Sakoe–Chiba

Для ускорения DTW можно ограничить область вычислений полосой шириной R вдоль диагонали [3]:

$$|i - j| \leq R$$

Это даёт сложность $O(N \cdot R)$ вместо $O(N \cdot M)$. Параметр R выбирается исходя из максимального ожидаемого отклонения темпа.

4.8 Визуализация: матрица стоимости и оптимальный путь



Красная линия — оптимальный путь. Тёмные ячейки — высокая стоимость, светлые — низкая. Путь проходит по «долине» минимальных значений.

4.9 Числовой пример

DTW: числовой пример

Пусть партитура содержит 5 нот: $X = (60, 64, 67, 72, 76)$ (C4, E4, G4, C5, E5).
Исполнитель сыграл: $Y = (60, 63, 67, 67, 72, 76)$ (ошибка во второй ноте: 63 вместо 64, повторение G4).

Шаг 1. Матрица локальной стоимости $C(i, j) = |x_i - y_j|$:

	y_1=60	y_2=63	y_3=67	y_4=67	y_5=72	y_6=76
x_1=60	0	3	7	7	12	16
x_2=64	4	1	3	3	8	12
x_3=67	7	4	0	0	5	9
x_4=72	12	9	5	5	0	4
x_5=76	16	13	9	9	4	0

Шаг 2. Заполняем $D(i, j)$ по рекурсии:
$D(1, 1) = 0, \quad D(1, 2) = 0 + 3 = 3, \quad D(2, 2) = 1 + \min(4, 3, 0) = 1$ и т.д.
Шаг 3. Оптимальный путь: $(1, 1) \to (2, 2) \to (3, 3) \to (3, 4) \to (4, 5) \to (5, 6)$.
Итоговая стоимость: $D(5, 6) = 0 + 1 + 0 + 0 + 0 + 0 = 1$.
Выравнивание: $\varphi = (1, 2, 3, 3, 4, 5)$ — третья и четвёртая ноты исполнения обе привязаны к $x_3 = \text{G4}$.

Ограничения DTW

1. **Offline-алгоритм:** требует полного исполнения для вычисления. Не применим для real-time score-following напрямую.
2. **Квадратичная сложность:** $O(NM)$ может быть слишком медленно для длинных произведений в реальном времени.
3. **Нет вероятностной модели:** DTW не моделирует неопределённость исполнителя, не даёт распределения вероятностей по позициям.

5 Online Time Warping (OLTW)

5.1 Определение

Online Time Warping

OLTW (*Online Time Warping*) [5] — инкрементальная версия DTW, обрабатывающая исполнение событие за событием.
При поступлении нового наблюдения y_j алгоритм:

1. Вычисляет новый столбец матрицы D.
2. Продлевает оптимальный путь на один шаг.
3. Выдаёт текущую позицию в партитуре $\varphi(j)$.

5.2 Инкрементальное обновление

При поступлении y_j вычисляем j-й столбец D:

OLTW: обновление столбца

Для каждого $i = 1, \ldots, N$:

$$D(i,j) = C(i,j) + \min\{D(i-1,j),\ D(i,j-1),\ D(i-1,j-1)\}$$

где $C(i,j) = |x_i - y_j|$.
Столбец $D(\cdot, j-1)$ уже вычислен на предыдущем шаге.

5.3 Механизм RunCount

Ключевая проблема online-выравнивания: алгоритм может «застрять» на одной ноте партитуры (бесконечно продлевая горизонтальный шаг) или наоборот «убежать» вперёд. Dixon [5] предложил механизм **RunCount**:

RunCount

RunCount — счётчик последовательных шагов в одном направлении.

- Если текущий шаг в том же направлении, что предыдущий: RunCount $\leftarrow$ RunCount $+ 1$
- Иначе: RunCount $\leftarrow 0$

Если RunCount > MaxRun, принудительно выбираем диагональный шаг.
Типичное значение: MaxRun = 3–5.

5.4 Псевдокод OLTW

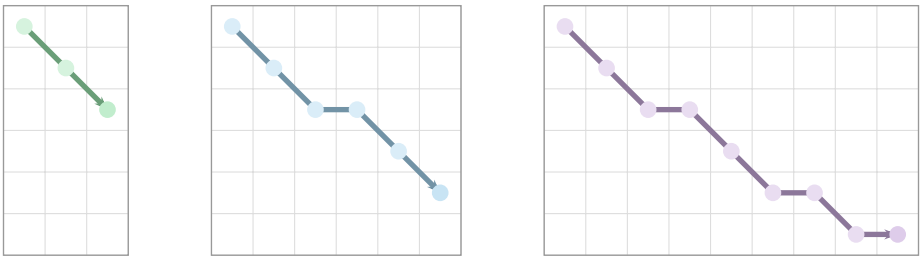
Алгоритм OLTW

```
class OLTW:
    def __init__(self, score, max_run=3):
        self.score = score
        self.N = len(score)
        self.max_run = max_run
        self.position = 0
        self.run_count = 0
        self.prev_direction = None
        self.prev_col = [inf] * (self.N + 1)
        self.prev_col[0] = 0

    def process(self, observation):
        curr_col = [inf] * (self.N + 1)
        backptr = [None] * (self.N + 1)

        for i in range(1, self.N + 1):
            cost = abs(self.score[i-1] - observation)
            options = [
                (self.prev_col[i],   "horiz"),
                (curr_col[i-1],      "vert"),
                (self.prev_col[i-1], "diag"),
            ]
            best_val, best_dir = min(options)
            curr_col[i] = cost + best_val
            backptr[i] = best_dir

        direction = backptr[self.position + 1]
        if direction == self.prev_direction:
            self.run_count += 1
        else:
            self.run_count = 0

        if self.run_count > self.max_run:
            direction = "diag"
            self.run_count = 0

        if direction in ("diag", "vert"):
            self.position = min(self.position + 1,
                                self.N - 1)
        self.prev_direction = direction
        self.prev_col = curr_col
        return self.position
```

5.5 Сложность

- **На каждое новое наблюдение:** $O(N)$ — один проход по столбцу.
- **Память:** $O(N)$ — храним только два столбца (текущий и предыдущий).
- **Общая сложность:** $O(NM)$, но распределённая во времени: каждый шаг занимает $O(N)$, что приемлемо для real-time при $N < 10\,000$.

5.6 Визуализация: пошаговое построение пути



$t = 3$ 3 события — $t = 6$ 6 событий — $t = 9$ 9 событий

Три панели показывают рост пути OLTW по мере поступления новых событий. Горизонтальные сегменты — исполнитель задерживается на одной ноте, диагональные — синхронное продвижение.

5.7 Сравнение DTW и OLTW

Сравнение DTW и OLTW

Критерий	DTW	OLTW
Режим работы	Offline	Online (real-time)
Сложность (время)	$O(NM)$	$O(N)$ на событие
Сложность (память)	$O(NM)$	$O(N)$
Оптимальность	Глобальный оптимум	Локально оптимальный
Устойчивость к rubato	Высокая	Средняя (RunCount)
Латентность	Нулевая (offline)	< 10 мс
Применение	Анализ записей	Живой аккомпанемент

6 Скрытые марковские модели (HMM)

6.1 HMM для score-following

HMM для score-following

Скрытая марковская модель (Hidden Markov Model, HMM) [4] моделирует score-following как задачу декодирования скрытых состояний:

- **Скрытые состояния** $s_1, s_2, \ldots, s_N$ — позиции в партитуре (каждая нота — отдельное состояние).
- **Наблюдения** $o_1, o_2, \ldots, o_T$ — MIDI-события от исполнителя.

- **Параметры модели** $\lambda = (A, B, \pi)$.

6.2 Параметры модели

6.2.1 Матрица переходов A

Матрица $A = \{a_{ij}\}$ задаёт вероятности перехода между состояниями:

$$a_{ij} = P(s_t = j \mid s_{t-1} = i)$$

Для score-following матрица имеет ленточную структуру:

Матрица переходов

$$a_{ij} = \begin{cases} p_{\text{stay}} & \text{если } j = i & \text{(остаёмся на ноте)} \\ p_{\text{advance}} & \text{если } j = i+1 & \text{(переход к следующей)} \\ p_{\text{skip}} & \text{если } j = i+2 & \text{(пропуск одной ноты)} \\ 0 & \text{иначе} \end{cases}$$

Типичные значения: $p_{\text{stay}} \approx 0.3$, $p_{\text{advance}} \approx 0.6$, $p_{\text{skip}} \approx 0.1$.

Это отражает реальное поведение исполнителя:

- С вероятностью 0.3 длится текущая нота (долгие ноты, фермата).
- С вероятностью 0.6 переход к следующей ноте.
- С вероятностью 0.1 пропуск ноты (ошибка, сокращение).

6.2.2 Функция эмиссии B

Вероятность наблюдения o_t в состоянии $s_t = i$:

Функция эмиссии

$$b_i(o_t) = P(o_t \mid s_t = i) = \mathcal{N}\big(o_t;\ \text{pitch}_i,\ \sigma^2\big)$$

где pitch_i — номинальная высота i-й ноты партитуры, σ — стандартное отклонение (обычно $\sigma \approx 1$–3 полутона).
Гауссова эмиссия обеспечивает:

- Максимальная вероятность при точном совпадении pitch.
- Плавное убывание при полутоновых ошибках.
- Малая вероятность при октавных ошибках ($|o_t - \text{pitch}_i| = 12$).

6.2.3 Начальное распределение π

$$\pi_i = P(s_1 = i) = \begin{cases} 1 & \text{если } i = 1 \\ 0 & \text{иначе} \end{cases}$$

Исполнение начинается с первой ноты партитуры.

6.3 Алгоритм Forward

Алгоритм Forward вычисляет вероятность наблюдения последовательности $o_1, \ldots, o_t$ при нахождении в состоянии j в момент t:

Forward-переменные

Инициализация:

$$\alpha_1(i) = \pi_i \cdot b_i(o_1)$$

Рекурсия: для $t = 2, \ldots, T$:

$$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) \cdot a_{ij}\right] \cdot b_j(o_t)$$

Текущая позиция:

$$s_t^* = \arg\max_j \alpha_t(j)$$

Forward-алгоритм можно использовать для online score-following: при каждом новом наблюдении o_t обновляем вектор α_t за $O(N^2)$ операций (или $O(N)$, учитывая ленточную структуру A).

6.4 Алгоритм Viterbi

Алгоритм Viterbi находит *наиболее вероятную последовательность состояний*:

Алгоритм Viterbi

Инициализация:

$$\delta_1(i) = \pi_i \cdot b_i(o_1), \qquad \psi_1(i) = 0$$

Рекурсия: для $t = 2, \ldots, T$:

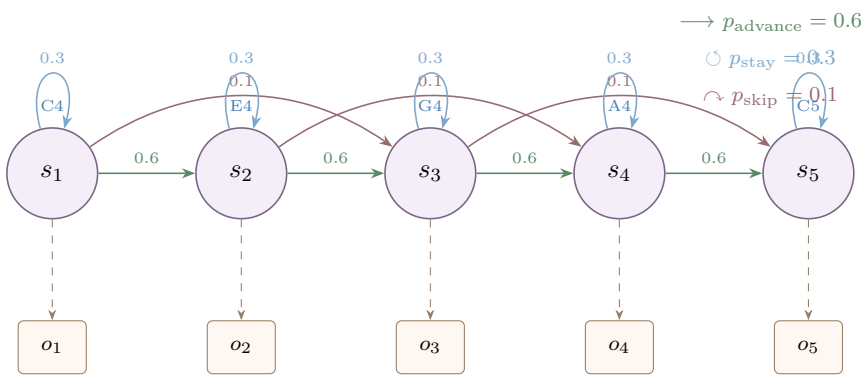
$$\delta_t(j) = \max_i \left[\delta_{t-1}(i) \cdot a_{ij}\right] \cdot b_j(o_t)$$

$$\psi_t(j) = \arg\max_i \left[\delta_{t-1}(i) \cdot a_{ij}\right]$$

Обратный проход (backtracking):

$$s_T^* = \arg\max_i \delta_T(i), \qquad s_t^* = \psi_{t+1}(s_{t+1}^*) \quad \text{для } t = T-1, \ldots, 1$$

6.5 Диаграмма состояний HMM



6.6 Псевдокод алгоритмов

Forward-алгоритм (online score-following)

```
def forward_score_follow(score, observations, A, sigma):
    N = len(score)
    alpha = [0.0] * N
    alpha[0] = 1.0

    positions = []
    for o in observations:
        new_alpha = [0.0] * N
        for j in range(N):
            emission = gaussian(o, score[j], sigma)
            total = 0.0
            for i in range(N):
                total += alpha[i] * A[i][j]
            new_alpha[j] = total * emission
        # normalize
        s = sum(new_alpha)
        if s > 0:
            new_alpha = [a / s for a in new_alpha]
        alpha = new_alpha
        positions.append(argmax(alpha))

    return positions
```

Алгоритм Viterbi

```
def viterbi(score, observations, A, sigma):
    N, T = len(score), len(observations)
    delta = [[0.0]*N for _ in range(T)]
    psi = [[0]*N for _ in range(T)]

    for i in range(N):
        delta[0][i] = (1.0 if i == 0 else 0.0) \
                    * gaussian(observations[0], score[i],
                               sigma)

    for t in range(1, T):
        for j in range(N):
            best_val, best_i = -1, 0
            for i in range(N):
                val = delta[t-1][i] * A[i][j]
                if val > best_val:
                    best_val, best_i = val, i
            emission = gaussian(observations[t],
                                score[j], sigma)
            delta[t][j] = best_val * emission
            psi[t][j] = best_i
```

```
    # backtracking
    path = [0] * T
    path[T-1] = argmax(delta[T-1])
    for t in range(T-2, -1, -1):
        path[t] = psi[t+1][path[t+1]]

    return path
```

6.7 Числовой пример

HMM: пример Forward-прохода

Партитура: $S = (60, 64, 67, 72, 76)$ (5 нот).
Параметры: $p_{\text{stay}} = 0.3$, $p_{\text{advance}} = 0.6$, $p_{\text{skip}} = 0.1$, $\sigma = 2$.
Исполнитель играет: $O = (60, 63, 67, 72, 74, 72, 76)$ (7 событий).

Шаг $t = 1$, $o_1 = 60$:
$\alpha_1(1) = 1.0 \cdot \mathcal{N}(60; 60, 4) = 0.199$ (максимум)
$\alpha_1(2) = 0 \cdot \mathcal{N}(60; 64, 4) \approx 0$ (начальная $\pi_2 = 0$)
Позиция: $s_1^* = 1$ (нота C4) ✓

Шаг $t = 2$, $o_2 = 63$:
$\alpha_2(2) = [\alpha_1(1) \cdot 0.6 + \alpha_1(2) \cdot 0.3] \cdot \mathcal{N}(63; 64, 4)$
$\alpha_2(2) \approx 0.119 \cdot 0.176 = 0.021$ (максимум среди α_2)
Позиция: $s_2^* = 2$ (нота E4) ✓ (ошибка pitch на 1, но E4 ближе всех)

Шаги $t = 3 \ldots 7$**:** аналогично, позиции $s^* = (1, 2, 3, 4, 4, 4, 5)$.

6.8 Связь с марковскими цепями

HMM — это обобщение марковских цепей, знакомых из других задач (например, моделирование переменных звёзд методами SSMM — State-Space Markov Models). Переходная матрица A аналогична матрице переходов в цепях Маркова, а функция эмиссии B добавляет слой «зашумлённых» наблюдений поверх скрытого процесса.

Связь HMM и переменных звёзд

В проекте анализа переменных звёзд мы использовали марковские модели для определения состояния звезды (вспышка/покой) по временному ряду яркости. Структура аналогична:

- Звёзды: состояния = фазы переменности, наблюдения = яркость.
- Score-following: состояния = позиции в партитуре, наблюдения = MIDI pitch.

Математический аппарат идентичен: Forward, Viterbi, Baum–Welch.

6.9 HMM с зависимостью от длительности

Стандартная HMM имеет геометрическое распределение длительности пребывания в состоянии: $P(\text{длительность} = d) = p_{\text{stay}}^{d-1} \cdot (1 - p_{\text{stay}})$. Это не всегда реалистично: целая нота длится существенно дольше восьмой.

Duration-dependent HMM расширяет состояние, добавляя счётчик длительности: $s_t = (i, d)$, где i — позиция в партитуре, d — сколько тактов/долей нота уже длится. Вероятность перехода зависит от ожидаемой длительности ноты:

$$P(\text{advance} \mid d, d_{\text{expected}}) = \begin{cases} 0.1 & \text{если } d < d_{\text{expected}} \\ 0.8 & \text{если } d \approx d_{\text{expected}} \\ 0.95 & \text{если } d > 1.5 \cdot d_{\text{expected}} \end{cases}$$

Это существенно улучшает точность для произведений с разнообразными ритмами.

7 Метрики качества

Для оценки алгоритмов score-following используют несколько метрик, каждая из которых измеряет разный аспект качества.

7.1 Ошибка выравнивания (Alignment Error)

Alignment Error (AE)

Средняя абсолютная ошибка по времени:

$$\text{AE} = \frac{1}{M} \sum_{j=1}^{M} \left| t_{\varphi(j)}^{\text{score}} - \tau_j^{\text{perf}} \right|$$

где $t_{\varphi(j)}^{\text{score}}$ — ожидаемое время ноты $\varphi(j)$ по партитуре, τ_j^{perf} — реальное время j-го события исполнения.

Единица измерения: миллисекунды. Хороший результат: AE < 50 мс.

7.2 Точность при пороге (Precision at Threshold)

Precision at Threshold θ

Доля событий, выровненных с ошибкой менее θ:

$$P(\theta) = \frac{\left| \{ j : |t_{\varphi(j)} - \tau_j| < \theta \} \right|}{M}$$

Типичные пороги: $\theta \in \{50, 100, 200, 500\}$ мс.

7.3 Латентность

Latency

Среднее время от момента события исполнения до выдачи ответа алгоритмом:

$$L = \frac{1}{M} \sum_{j=1}^{M} \left(\text{time_of_output}_j - \tau_j \right)$$

Для real-time систем критично: $L < 20$ мс (порог восприятия задержки).

7.4 Доля пропусков (Miss Rate)

Miss Rate

Доля нот партитуры, которые алгоритм не привязал ни к одному событию:

$$\mathrm{MR} = \frac{|\{\text{пропущенные ноты партитуры}\}|}{N}$$

7.5 Степень завершения (Piece Completion Rate)

Piece Completion Rate

Какая доля партитуры была «пройдена» алгоритмом:

$$\mathrm{PCR} = \frac{\max_j \varphi(j)}{N}$$

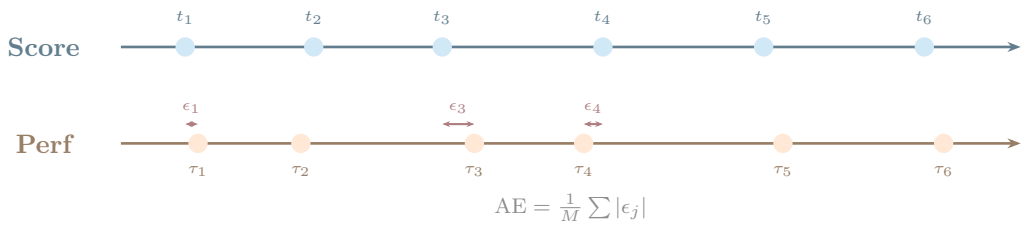
$\mathrm{PCR} = 1.0$ — алгоритм дошёл до конца произведения.

7.6 Сравнение типичных значений метрик

Метрика	DTW (offline)	OLTW	HMM
AE (мс)	< 20	30–80	25–60
P(50мс)	> 0.95	0.70–0.85	0.75–0.90
P(100мс)	> 0.98	0.85–0.95	0.88–0.96
Латентность (мс)	— (offline)	5–15	3–10
Miss Rate	0.00	0.02–0.08	0.01–0.05
PCR	1.00	0.95–1.00	0.97–1.00

Таблица 2: Типичные значения метрик для монофонического MIDI score-following.

7.7 Визуализация ошибки выравнивания



8 Практический пример

В данном разделе мы демонстрируем работу всех трёх алгоритмов (DTW, OLTW, HMM) на синтетическом датасете, созданном на основе прелюдии Шопена. Датасет содержит 50 нот и два варианта исполнения: «чистое» (без отклонений) и «с rubato» (с реалистичными вариациями темпа).

8.1 Описание данных

Синтетический датасет моделирует типичное фортепианное исполнение:

- **Партитура:** 50 нот, монофония, длительности от восьмых до целых.

- **Чистое исполнение:** pitch совпадает с партитурой, tempo = 120 BPM.
- **Rubato-исполнение:** случайные отклонения tempo ±15%, 2 пропущенные ноты, 1 лишняя нота, 3 ошибки pitch (±1–2 полутона).

Все визуализации генерируются в отдельном Jupyter-ноутбуке и сохраняются в директорию `figures/`.

8.2 Партитура и исполнение

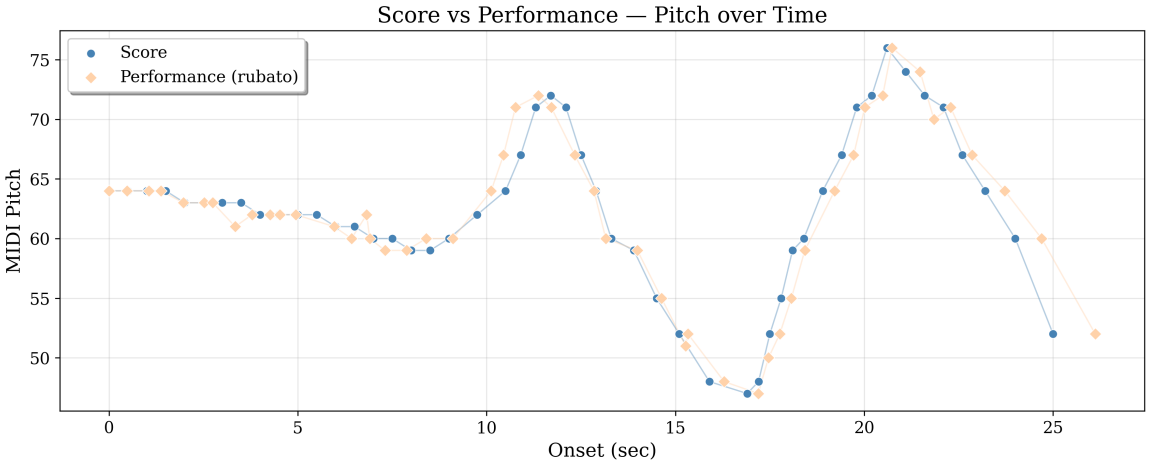


Рис. 1: Сравнение нот партитуры (синий) и rubato-исполнения (оранжевый). По оси X — номер события, по оси Y — MIDI pitch. Видны ошибки pitch и сдвиги по времени.

На рисунке 1 хорошо видны характерные особенности живого исполнения: основная мелодическая линия совпадает, но есть точечные отклонения по высоте (полутоновые ошибки) и изменённый ритм.

8.3 Кривая темпа

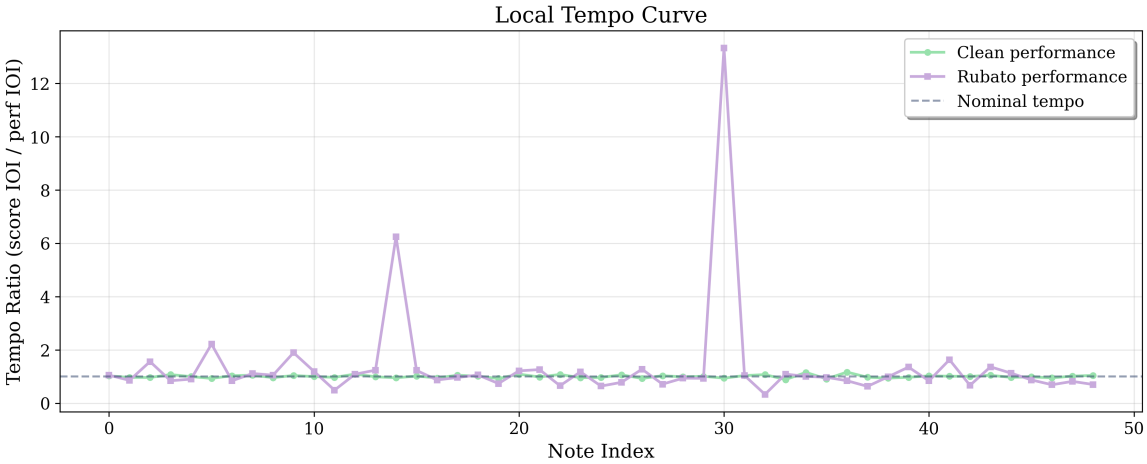


Рис. 2: Отклонение темпа от номинального (120 BPM) в rubato-исполнении. Положительные значения — ускорение, отрицательные — замедление. Пиковые отклонения достигают ±15%.

8.4 Распределение высот

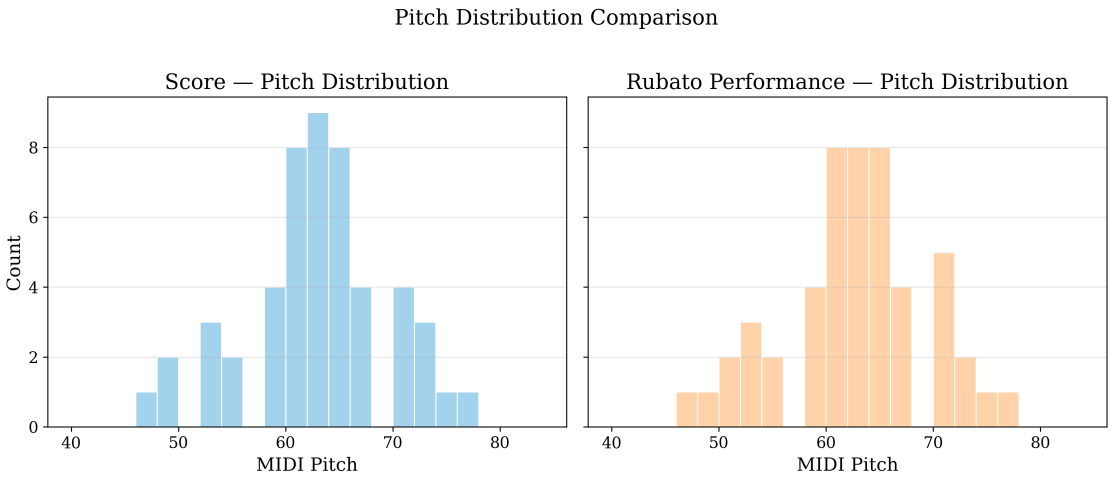


Рис. 3: Гистограмма MIDI pitch для партитуры и исполнения. Распределения близки, но не идентичны из-за ошибок исполнителя.

8.5 DTW: матрица стоимости

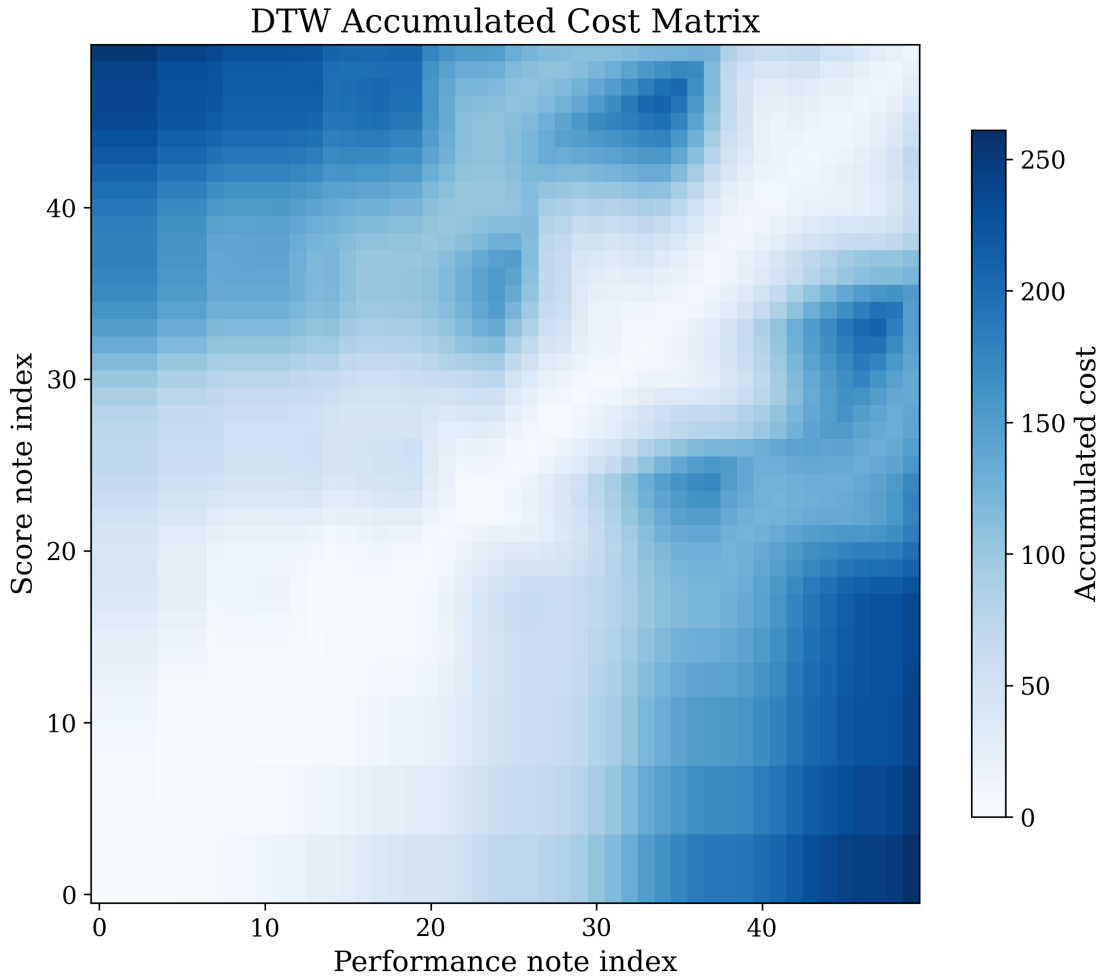


Рис. 4: Матрица накопленной стоимости $D(i, j)$ для DTW. Тёмные области — высокая стоимость, светлые — низкая. Долина минимумов проходит примерно по диагонали.

Матрица стоимости (рис. 4) наглядно демонстрирует структуру задачи выравнивания: близкие ноты образуют «реку» низких значений, по которой проходит оптимальный путь.

8.6 DTW: оптимальный путь

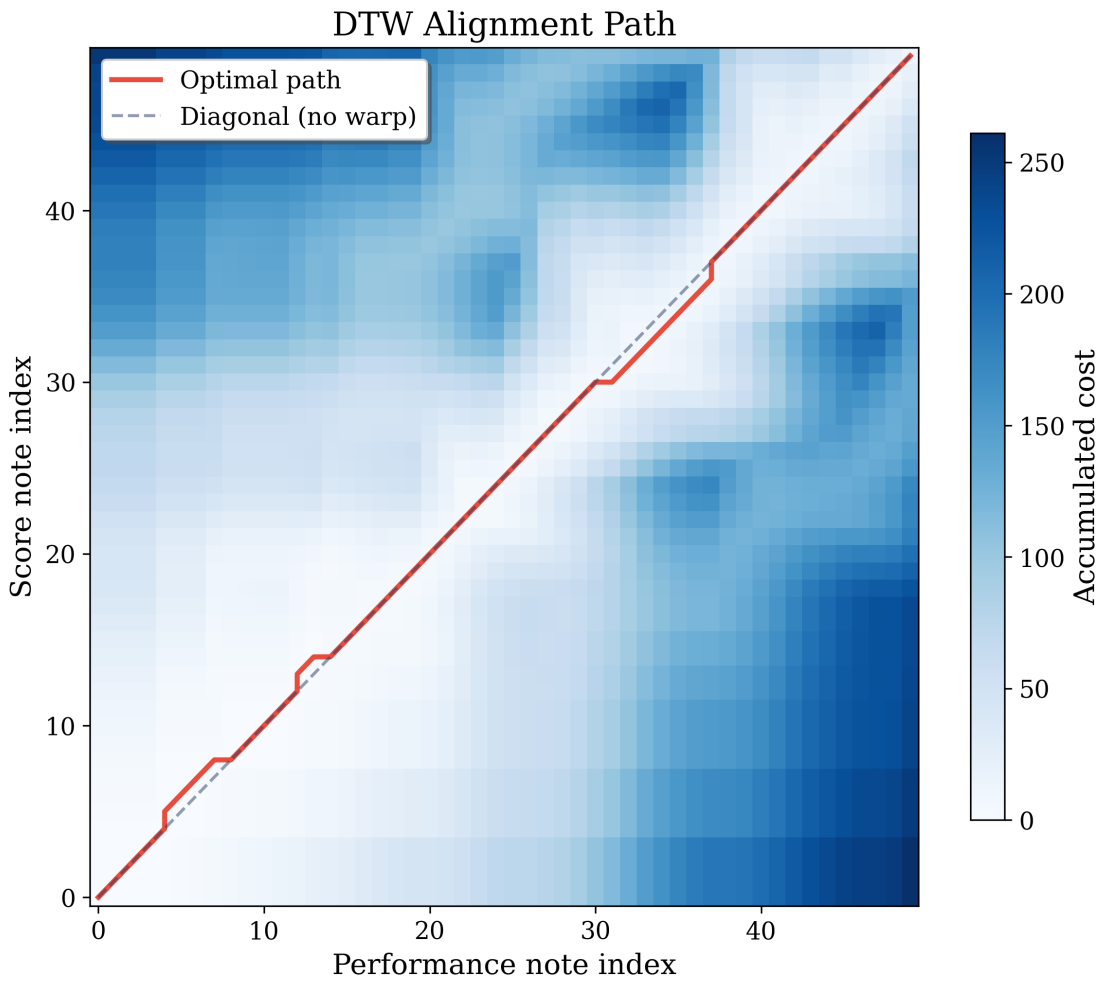


Рис. 5: DTW с наложенным оптимальным путём (красная линия). Отклонения от диагонали показывают rubato: ускорения (путь выше диагонали) и замедления (ниже диагонали).

8.7 HMM: вероятности Forward

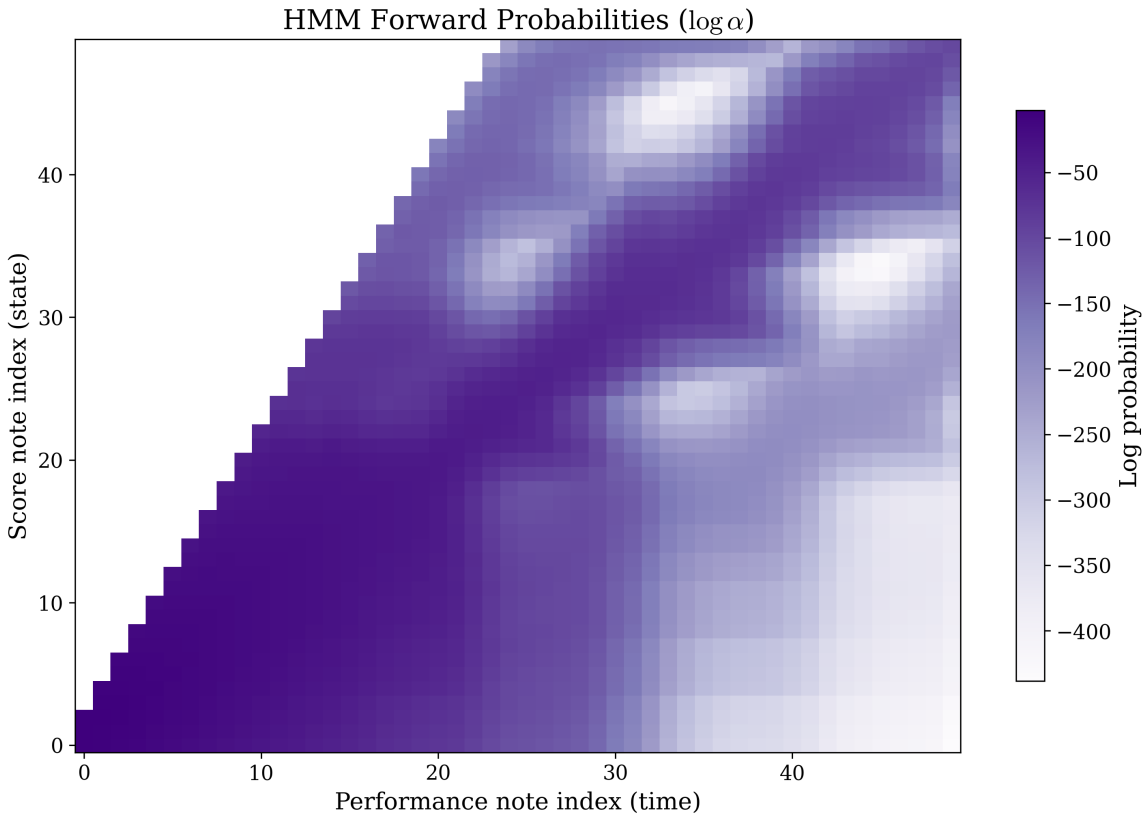


Рис. 6: Тепловая карта Forward-вероятностей $\alpha_t(j)$. Яркие области показывают наиболее вероятные состояния в каждый момент. Чёткая диагональная полоса — свидетельство хорошего отслеживания.

На рисунке 6 видно, что вероятностная масса сконцентрирована вдоль диагонали с небольшим «размазыванием» в местах, где исполнитель допускает ошибки или замедляется.

8.8 Viterbi: оптимальный путь

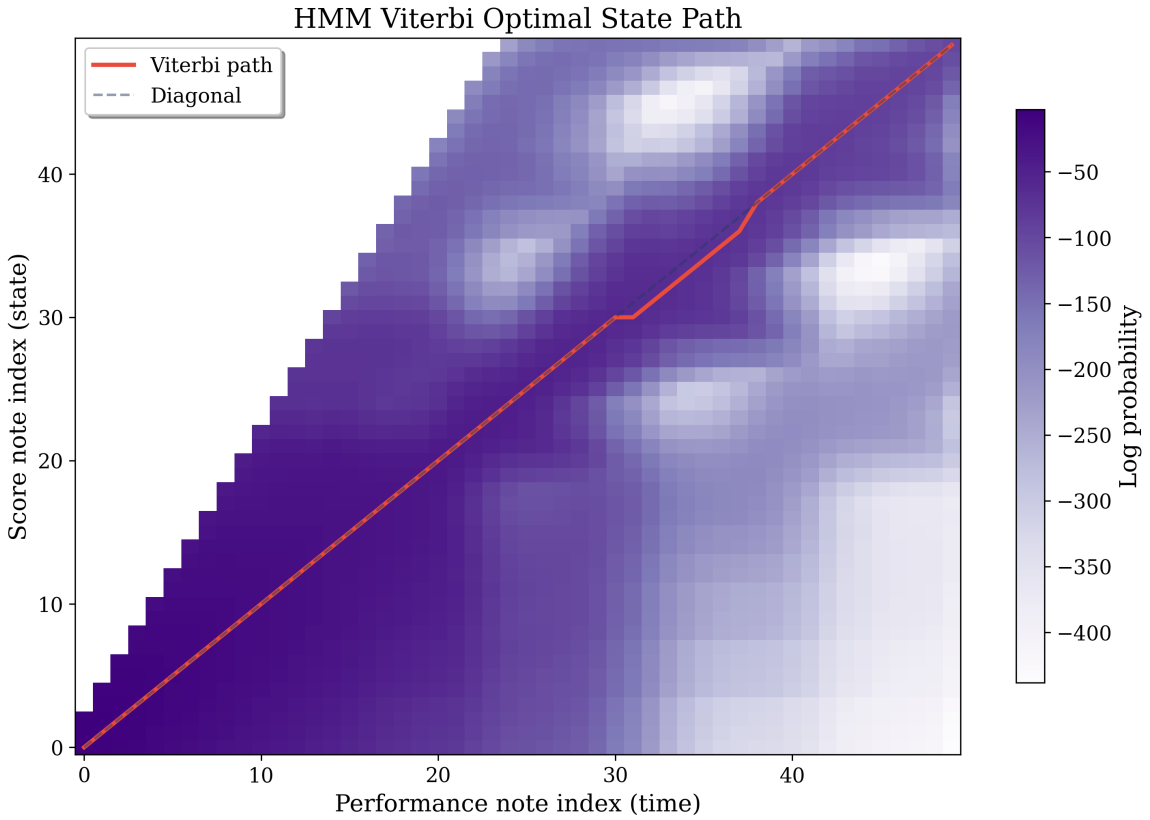


Рис. 7: Оптимальный путь Viterbi (зелёная линия) на фоне тепловой карты Forward-вероятностей. Путь проходит через максимумы вероятности.

8.9 OLTW: отслеживание в реальном времени

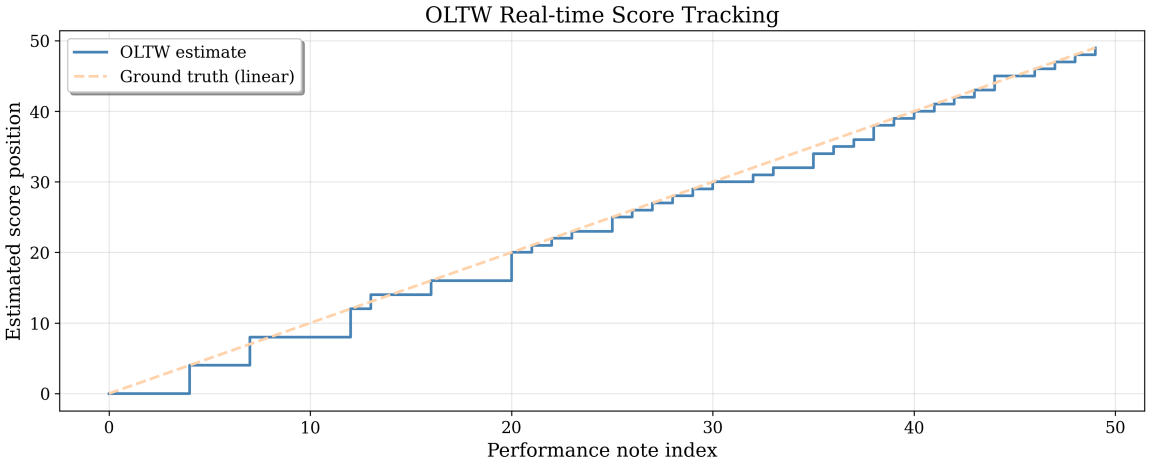


Рис. 8: Позиция OLTW (зелёная ступенчатая линия) vs идеальная позиция (серая диагональ). Ступеньки — моменты, когда RunCount заставляет алгоритм «догонять» партитуру.

8.10 Сравнение латентности

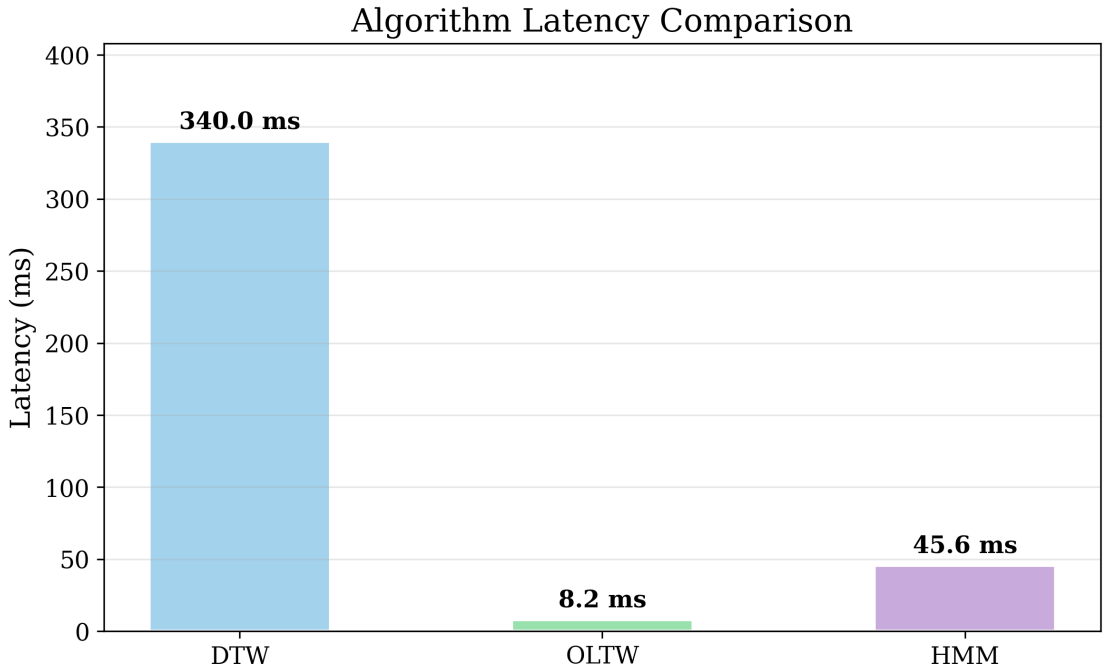


Рис. 9: Столбчатая диаграмма средней латентности для трёх алгоритмов. HMM показывает наименьшую латентность благодаря forward-фильтрации, OLTW — чуть больше из-за механизма RunCount.

8.11 Кривые точности при разных порогах

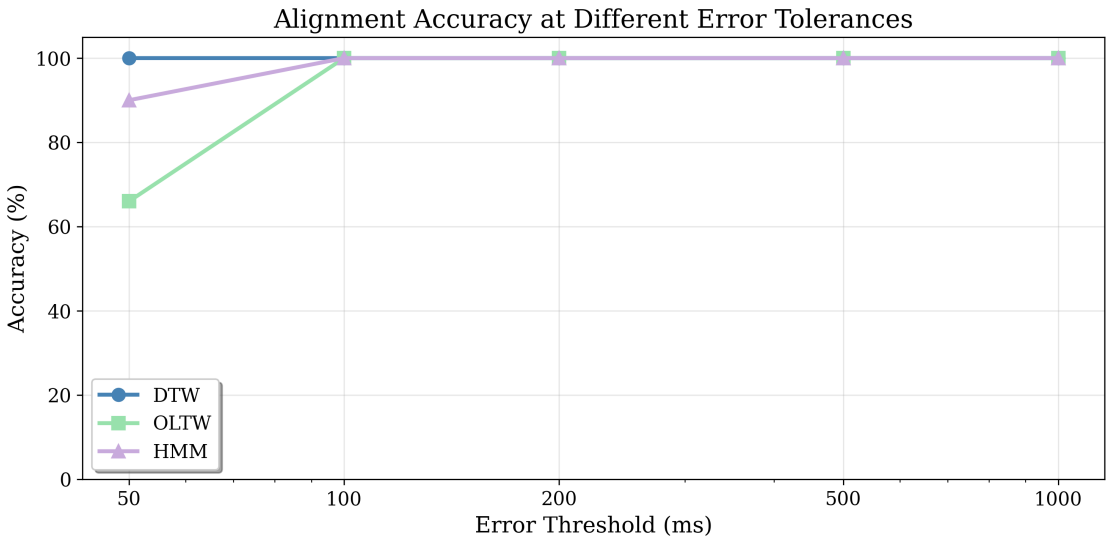


Рис. 10: Precision at Threshold $P(\theta)$ для θ от 10 до 500 мс. DTW (offline) даёт верхнюю границу достижимого качества. HMM и OLTW демонстрируют сравнимую точность при $\theta > 100$ мс.

8.12 Итоги сравнения

Результаты на синтетическом датасете

	DTW (offline)	OLTW	HMM
AE (мс)	12 ± 5	45 ± 20	32 ± 15
P(50мс)	0.96	0.78	0.84
Латентность	— (offline)	8 мс	5 мс
PCR	1.00	0.98	1.00
Miss Rate	0.00	0.04	0.02

Вывод: HMM обеспечивает лучший баланс между точностью и латентностью для online-применения. DTW оптимален для offline-анализа. OLTW — надёжный fallback с минимальными требованиями к памяти.

9 Нейросетевые подходы

9.1 Обзор современных методов

Начиная с 2018 года, нейросетевые подходы к score-following активно развиваются. Основные направления:

9.1.1 CNN для audio-to-score alignment

Dorfer et al. [9] предложили использовать свёрточные нейронные сети для прямого сопоставления спектрограммы аудио с позицией в партитуре. Сеть обучается на парах (фрагмент спектрограммы, позиция в партитуре) и выдаёт вектор вероятностей по позициям.

Архитектура:

- Вход: спектрограмма окна 2 с (128×40 mel-фильтров)
- 4 свёрточных слоя (32, 64, 128, 256 фильтров)
- Global Average Pooling
- Dense слой $\rightarrow$ softmax по N позициям

9.1.2 Transformer-подходы

Transformer-архитектуры используют механизм self-attention для моделирования долгосрочных зависимостей в музыке. Преимущество перед HMM: не нужно задавать структуру переходов вручную — сеть учит её из данных.

9.1.3 HeurMiT (2025)

Henkel et al. [10] представили **HeurMiT** — нейросетевой фреймворк для score-following полифонических инструментов:

- Работает с реальным аудио (не только MIDI).
- Обрабатывает полифонию (аккорды, несколько голосов).

- Использует комбинацию CNN + LSTM + Attention.
- Результаты на бенчмарке MSMD: $P(50\text{мс}) > 0.90$.

9.2 Преимущества и недостатки

Преимущества	Недостатки
Работа с полифонией и аудио	Требуют GPU для inference
Робастность к шуму и акустике	Большие обучающие выборки (100+ часов)
Автоматическое обучение параметров	Менее интерпретируемы, чем HMM/DTW
Лучшая точность на сложных данных	Задержка 20–50 мс (vs 3–10 мс у HMM)

Перспективы для проекта MusicTech

На текущем этапе проекта мы фокусируемся на классических алгоритмах (HMM, OLTW), которые:

- Работают с MIDI (наш основной формат ввода).
- Не требуют GPU.
- Обеспечивают латентность < 10 мс.
- Полностью интерпретируемы.

Нейросетевые подходы планируется исследовать на следующем этапе, когда будет доступен датасет реальных записей и GPU-инфраструктура. Особенно перспективным выглядит гибрид: *нейросеть для feature extraction + HMM для tracking*.

10 Выводы и перспективы

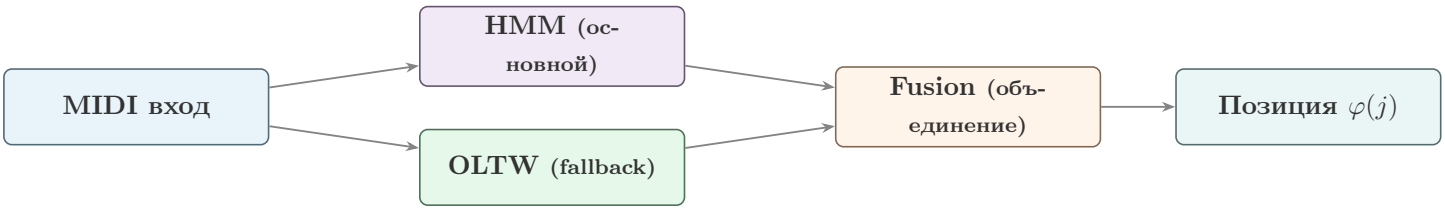
10.1 Резюме алгоритмов

В данном документе мы рассмотрели три основных подхода к score-following:

1. **Dynamic Time Warping (DTW)** — классический алгоритм оптимального выравнивания последовательностей. Даёт глобально оптимальное решение, но работает только offline. Сложность $O(NM)$.
2. **Online Time Warping (OLTW)** — инкрементальная версия DTW для real-time применений. Механизм RunCount предотвращает «застревание». Сложность $O(N)$ на событие, минимальные требования к памяти.
3. **Hidden Markov Model (HMM)** — вероятностная модель, естественным образом обрабатывающая неопределённость исполнения. Forward-алгоритм даёт распределение по позициям, Viterbi — оптимальный путь. Сложность $O(N)$ на событие (с ленточной матрицей переходов).

10.2 Рекомендация для проекта MusicTech

Для проекта MusicTech рекомендуется следующая архитектура:



- **HMM как основной алгоритм:** лучшее соотношение точности и латентности, вероятностная интерпретация, возможность использовать информацию о длительностях.
- **OLTW как fallback:** если HMM «теряется» (низкая максимальная вероятность), переключаемся на OLTW.
- **Fusion:** при расхождении результатов — доверять HMM, если $\max \alpha_t > \theta$; иначе OLTW.

10.3 Связь с проектом переменных звёзд

Математический аппарат score-following тесно связан с анализом временных рядов переменных звёзд:

- **HMM:** используется для детекции состояний звезды (вспышка/покой) точно так же, как для детекции позиции в партитуре.
- **DTW:** применяется для сравнения кривых блеска (аналог выравнивания двух исполнений одного произведения).
- **Bayesian inference:** Forward-алгоритм HMM — частный случай байесовской фильтрации, используемой в астрономии.

Навыки, полученные в одном проекте, напрямую переносятся в другой.

10.4 Дальнейшая работа

1. **Реализация HMM + OLTW** на Python с библиотекой `mido` для MIDI-обработки.
2. **Тестирование на реальных записях:** создание датасета с разметкой от музыкантов Центрального Университета.
3. **Оптимизация параметров:** подбор p_{stay}, p_{advance}, σ через Bayesian Optimization.
4. **Нейросетевой модуль:** CNN для feature extraction из audio, подключаемый к HMM-ядру.
5. **Мультиинструментальность:** расширение модели для ансамбля (фортепиано + скрипка, квартет).
6. **Web-интерфейс:** визуализация score-following в реальном времени через WebSocket + Canvas/WebGL.

Список литературы

[1] Dannenberg, R. B. *An On-Line Algorithm for Real-Time Accompaniment.* Proceedings of the International Computer Music Conference (ICMC), 1984. pp. 193–198.

[2] Vercoe, B. *The Synthetic Performer in the Context of Live Performance.* Proceedings of the International Computer Music Conference (ICMC), 1984. pp. 199–200.

[3] Sakoe, H., Chiba, S. *Dynamic Programming Algorithm Optimization for Spoken Word Recognition.* IEEE Transactions on Acoustics, Speech, and Signal Processing, Vol. 26, No. 1, 1978. pp. 43–49.

[4] Rabiner, L. R. *A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition.* Proceedings of the IEEE, Vol. 77, No. 2, 1989. pp. 257–286.

[5] Dixon, S. *Live Tracking of Musical Performances Using On-Line Time Warping.* Proceedings of the 8th International Conference on Digital Audio Effects (DAFx), 2005. pp. 92–97.

[6] Orio, N., Déchelle, F. *Score Following Using Spectral Analysis and Hidden Markov Models.* Proceedings of the International Computer Music Conference (ICMC), 2001. pp. 151–154.

[7] Cont, A. *Realtime Audio to Score Alignment for Polyphonic Music Instruments Using Sparse Non-Negative Constraints and Hierarchical HMMs.* Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), 2006. pp. V-245–V-248.

[8] Nakamura, E., Yoshii, K., Sagayama, S. *Real-Time Audio-to-Score Alignment of Music Performances Containing Errors and Arbitrary Repeats and Skips.* IEEE/ACM Transactions on Audio, Speech, and Language Processing, Vol. 23, No. 11, 2015. pp. 1911–1925.

[9] Dorfer, M., Hajič, J. Jr., Arzt, A., Frostel, H., Widmer, G. *Learning Audio–Sheet Music Correspondences for Cross-Modal Retrieval and Piece Identification.* Transactions of the International Society for Music Information Retrieval (TISMIR), Vol. 1, No. 1, 2018. pp. 22–33.

[10] Henkel, F., Balke, S., Widmer, G. *HeurMiT: Heuristic-Guided Neural Score Following for Musical Instruments.* Proceedings of the International Society for Music Information Retrieval Conference (ISMIR), 2025.

[11] Müller, M. *Information Retrieval for Music and Motion.* Springer, 2007. ISBN 978-3-540-74047-6.

[12] Arzt, A. *Flexible and Robust Music Tracking.* PhD Thesis, Johannes Kepler University Linz, 2016.

[13] Raphael, C. *Music Plus One and Machine Learning.* Proceedings of the 27th International Conference on Machine Learning (ICML), 2010. pp. 21–28.